SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 - CONCEPT MAPPING

Course Code:	CSA1216	**Course Title:**	Computer Architecture
CO Assessed:	**CO1**	**Assessment :**	**ASSESSMENT TOOL 2- CONCEPT** MAPPING
Weightage:	**35%**	**Total Marks:**	**25**
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	**Jana Malakondaiah**	**Reg.No.:**	**192424223**
Department:	**AI & DS**	**Date:**	**23/07/2026**

Instruction to Students

1. Each question carries 5 mark.
2. Only the appropriate Tools can be used
3. Time allotted for the exam is 60 min

Code of Conduct:

I Jana Malakondaiah(192424223) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Jana Malakondaiah
Signature of the student:

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

1. CPU, Memory, I/O Devices and Bus Structures

Central Topic
Computer System

Organization
Main Concepts

- CPU
- Memory
- Input/Output (I/O) Devices
- Bus Structures

Sub-concepts
CPU

- Arithmetic Logic Unit (ALU)
- Control Unit (CU)
- Registers

Memory

- Cache Memory
- RAM (Primary Memory)

I/O Devices

- Input Devices (Keyboard, Mouse, Scanner)
- Output Devices (Monitor, Printer, Speaker)

Bus Structures

- Single Bus
- Multi Bus
- Hierarchical Bus

Relationships (Linking Words)

- CPU is composed of the ALU, Control Unit, and Registers.
- ALU executes arithmetic and logical calculations.
- Control Unit coordinates the execution of instructions.
- Registers hold temporary data, addresses, and instructions.
- Cache Memory keeps frequently accessed information for faster processing.
- RAM stores active programs and user data.
- System Bus carries data, addresses, and control signals between components.
- Single Bus supports communication through one common path.
- Multi Bus enables simultaneous data transfers.
- Hierarchical Bus enhances system scalability and communication efficiency.
- I/O Devices communicate with the CPU and memory through the bus system.

Hierarchy (General → Specific)
Computer System
→ CPU, Memory, I/O Devices, Bus Structures
→ CPU (ALU, Control Unit, Registers)
→ Memory (Cache Memory, RAM)

→ I/O Devices (Input Devices, Output Devices)
→ Bus Structures (Single Bus, Multi Bus, Hierarchical Bus)

Cross-links

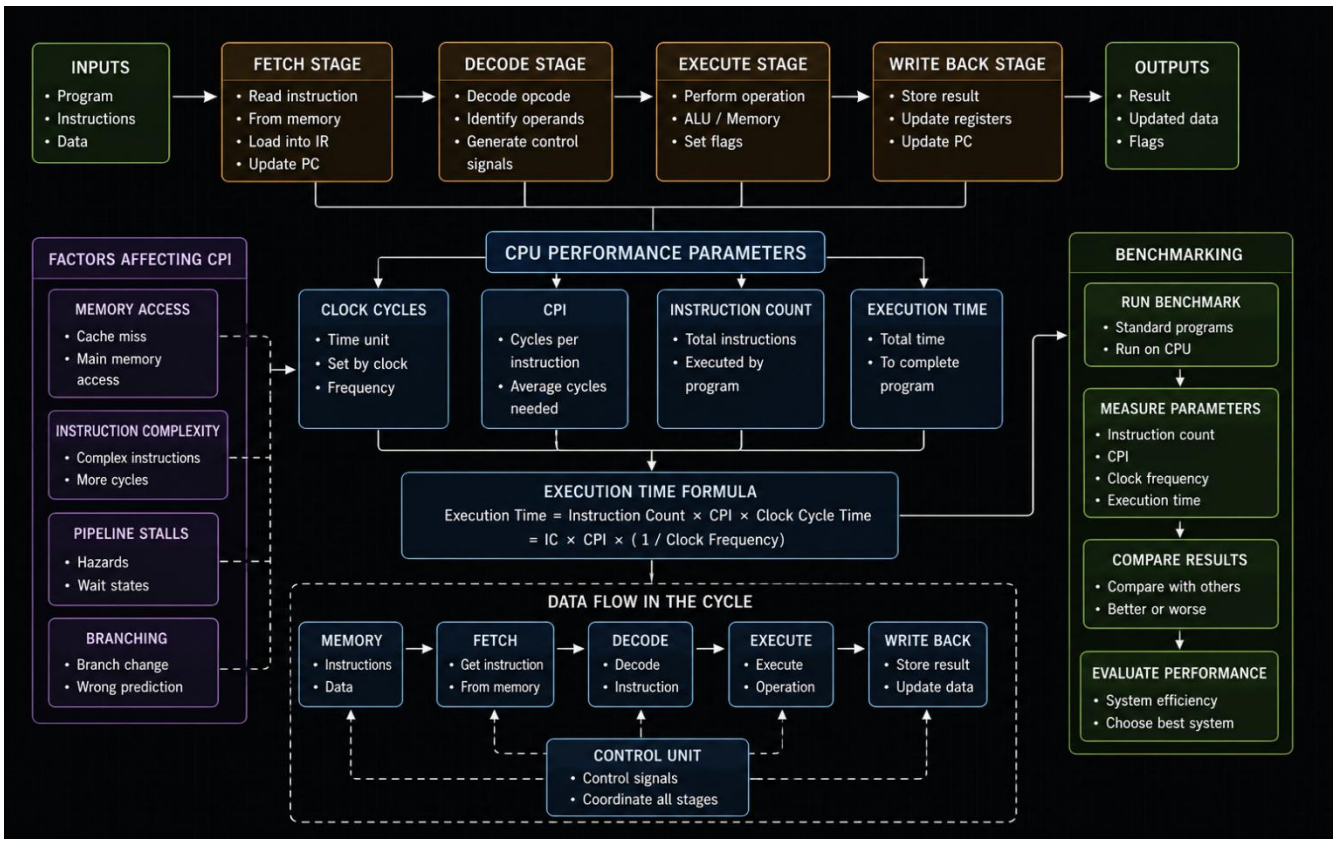
- Cache Memory improves CPU performance by decreasing memory access time.
- Registers exchange data with the ALU during instruction processing.
- Bus Structures determine the speed of communication between hardware components.
- Multi Bus and Hierarchical Bus increase system efficiency by reducing communication delays.

Real-world Application

Desktop computers, laptops, smartphones, servers, embedded systems, and industrial automation systems.

Summary / Inference

A well-designed computer system depends on efficient interaction among the CPU, memory, I/O devices, and bus structures. Advanced bus architectures and memory hierarchy reduce communication delays, increase processing speed, and improve the overall performance of modern computing systems.



2. Instruction Execution Cycle and CPU Performance

Central Topic
Instruction Execution Cycle
Organization
Main Concepts

- Fetch
- Decode
- Execute
- CPU Performance

Sub-concepts
Fetch

- Retrieve instruction from memory
- Load instruction into the Instruction Register (IR)

Decode

- Decode the opcode
- Identify source and destination operands

Execute

- Carry out the required operation
- Write the result to a register or memory

CPU Performance

- CPI (Cycles Per Instruction)
- Clock Cycles
- Execution Time
- Benchmarking

Relationships (Linking Words)

- Fetch retrieves the instruction from main memory.
- Decode analyzes the opcode and identifies the required operands.
- Execute completes the specified operation using the ALU.
- Memory Access influences the time required for the fetch stage.
- Instruction Complexity affects the execution process.
- Pipeline Stalls increase the number of Cycles Per Instruction (CPI).
- Clock Cycles determine how quickly instructions are processed.
- Benchmarking measures and compares processor performance.

Hierarchy (General → Specific)
Instruction Execution Cycle
→ Fetch, Decode, Execute
→ CPU Performance
→ CPI, Clock Cycles, Execution Time, Benchmarking
Cross-links

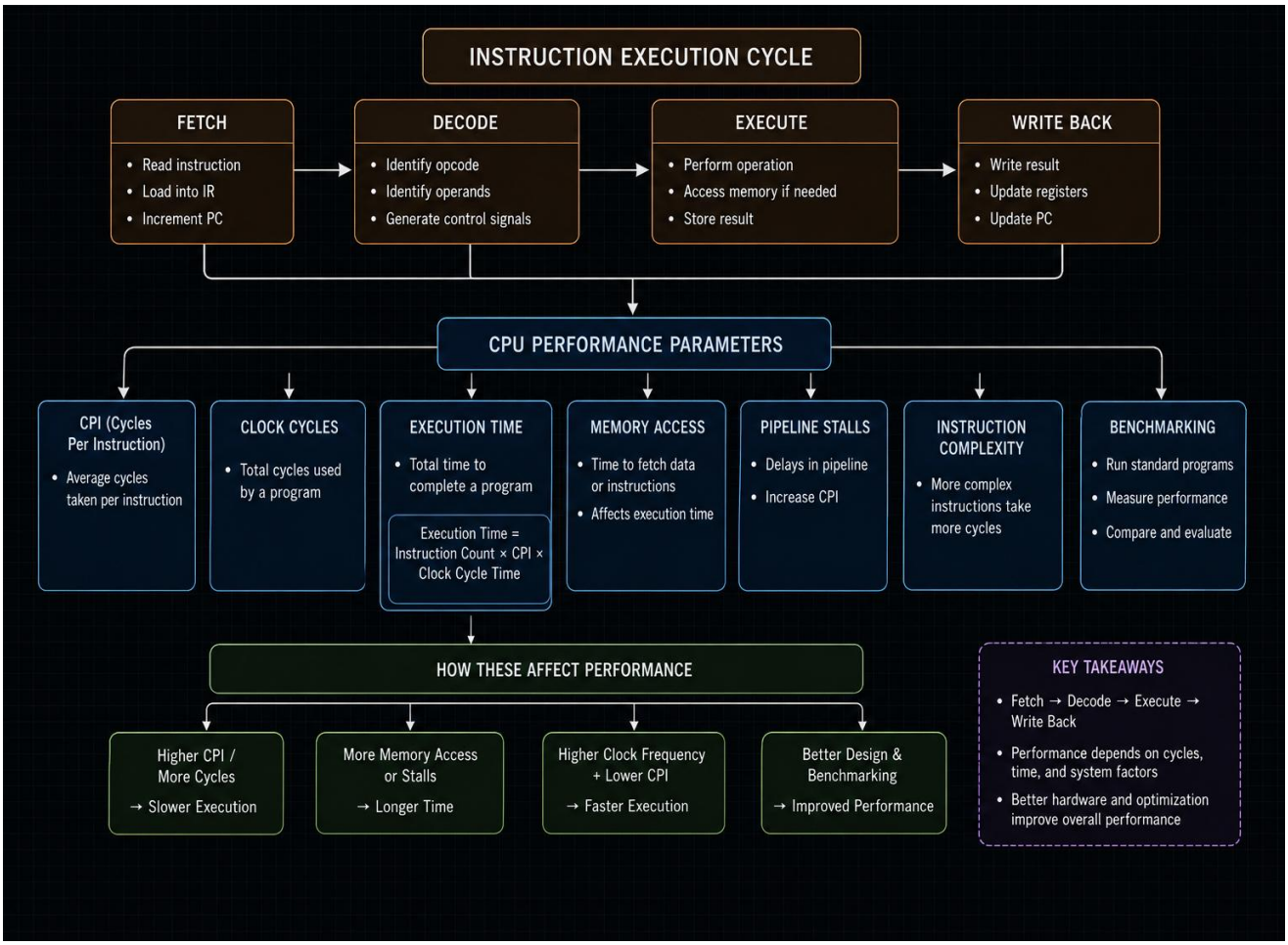
- Faster memory access reduces instruction execution time.
- Pipeline stalls lower processor efficiency by increasing CPI.
- Clock frequency improves instruction processing speed.
- Benchmarking compares the performance of different processors using standard tests.

Real-world Application

Performance evaluation of Intel, AMD, ARM, Apple Silicon processors, data centers, and high-performance computing systems.

Summary / Inference:

The overall performance of a processor depends on the efficient execution of the Fetch, Decode, and Execute stages. Faster memory access, lower CPI, fewer pipeline stalls, and optimized clock cycles improve execution speed, while benchmarking helps evaluate and compare processor performance.



3. Comparison of 8085 and 8086 Architectures

Central Topic

8085 vs 8086 Architecture

Main Concepts

- Instruction Set
- Register Organization
- Addressing Modes
- Memory Segmentation

Sub-concepts

8085

- 8-bit processor
- 64 KB memory
- Six general-purpose registers
- Limited addressing modes

8086

- 16-bit processor
- 1 MB memory
- General-purpose and segment registers
- Memory Segmentation
- Advanced addressing modes

Relationships (Linking Words)

- 8086 supports larger memory than 8085.
- Memory Segmentation improves memory management.
- Registers increase execution efficiency.
- Advanced Addressing Modes improve flexibility.
- Instruction Set determines processing capability.
- Segment Registers manage code, data, stack, and extra memory.

Hierarchy (General → Specific)

Microprocessor Architecture → 8085 and 8086 → Registers, Instruction Set, Addressing Modes, Memory Segmentation.

Cross-links

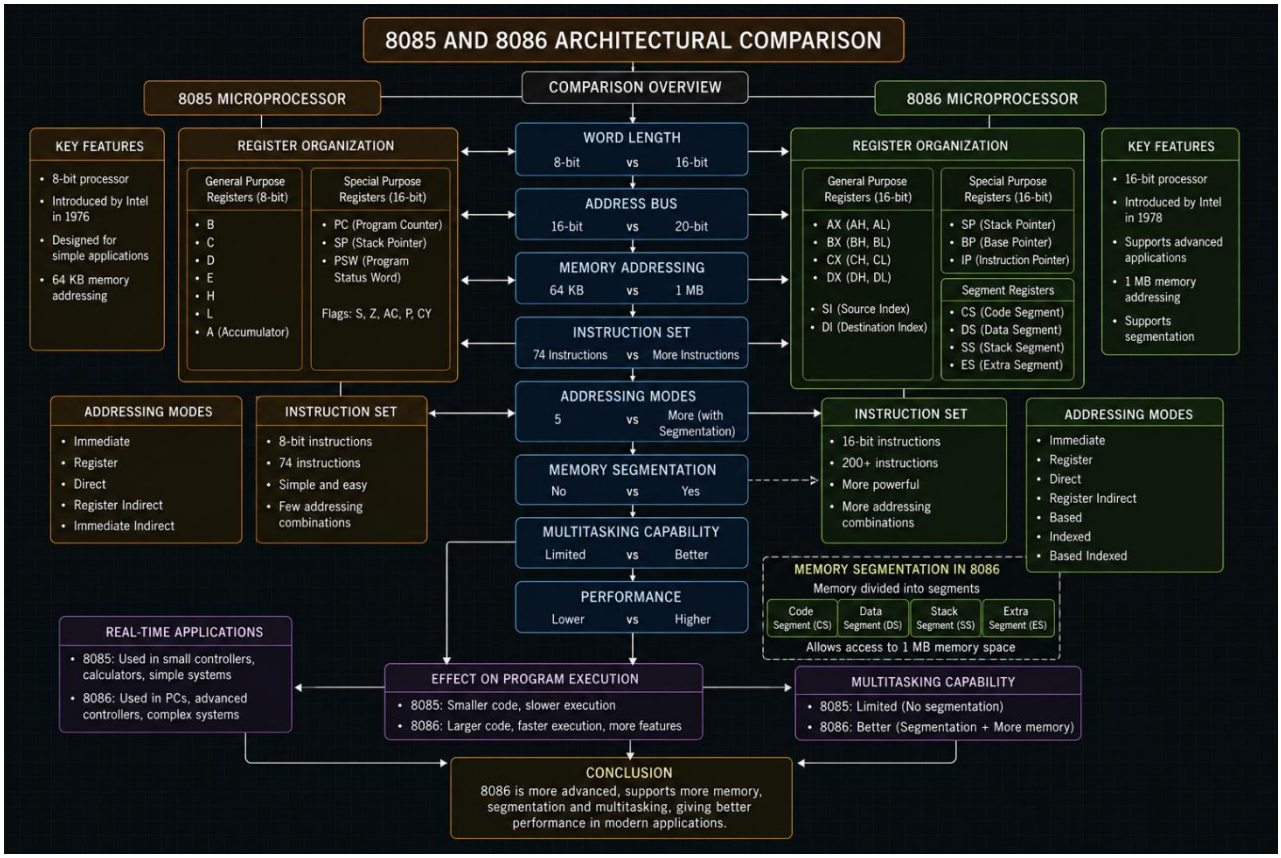
- Memory Segmentation supports multitasking.
- More Registers increase execution speed.
- Advanced Instructions improve real-time performance.

Real-world Application

Industrial automation, embedded systems, robotics, and control systems.

Summary/Inference

The 8086 architecture provides higher speed, larger memory, better multitasking, and greater efficiency than the 8085 architectures.


8085 AND 8086 ARCHITECTURAL COMPARISON
COMPARISON OVERVIEW
8085 MICROPROCESSOR
8086 MICROPROCESSOR
KEY FEATURES
8-bit processor
Introduced by Intel in 1976
Designed for simple applications
64 KB memory addressing
REGISTER ORGANIZATION
General Purpose Registers (8-bit)
B
C
D
E
H
L
A (Accumulator)
Special Purpose Registers (16-bit)
PC (Program Counter)
SP (Stack Pointer)
PSW (Program Status Word)
Flags: S, Z, AC, P, CY
WORD LENGTH
8-bit vs 16-bit
ADDRESS BUS
16-bit vs 20-bit
MEMORY ADDRESSING
64 KB vs 1 MB
INSTRUCTION SET
74 Instructions vs More Instructions
ADDRESSING MODES
5 vs More (with Segmentation)
MEMORY SEGMENTATION
No vs Yes
MULTITASKING CAPABILITY
Limited vs Better
PERFORMANCE
Lower vs Higher
REGISTER ORGANIZATION
General Purpose Registers (16-bit)
AX (AH, AL)
BX (BH, BL)
CX (CH, CL)
DX (DH, DL)
SI (Source Index)
DI (Destination Index)
Special Purpose Registers (16-bit)
SP (Stack Pointer)
BP (Base Pointer)
IP (Instruction Pointer)
Segment Registers
CS (Code Segment)
DS (Data Segment)
SS (Stack Segment)
ES (Extra Segment)
KEY FEATURES
16-bit processor
Introduced by Intel in 1978
Supports advanced applications
1 MB memory addressing
Supports segmentation
ADDRESSING MODES
Immediate
Register
Direct
Register Indirect
Immediate Indirect
INSTRUCTION SET
8-bit instructions
74 instructions
Simple and easy
Few addressing combinations
INSTRUCTION SET
16-bit instructions
200+ instructions
More powerful
More addressing combinations
ADDRESSING MODES
Immediate
Register
Direct
Register Indirect
Based
Indexed
Based Indexed
MEMORY SEGMENTATION IN 8086
Memory divided into segments
Code Segment (CS)
Data Segment (DS)
Stack Segment (SS)
Extra Segment (ES)
Allows access to 1 MB memory space
REAL-TIME APPLICATIONS
8085: Used in small controllers, calculators, simple systems
8086: Used in PCs, advanced controllers, complex systems
EFFECT ON PROGRAM EXECUTION
8085: Smaller code, slower execution
8086: Larger code, faster execution, more features
MULTITASKING CAPABILITY
8085: Limited (No segmentation)
8086: Better (Segmentation + More memory)
CONCLUSION
8086 is more advanced, supports more memory, segmentation and multitasking, giving better performance in modern applications.

4. Addressing Modes and Instruction Types

Central Topic
Addressing Modes
Organization
Main Concepts

- Immediate Addressing
- Direct Addressing
- Register Addressing
- Indirect Addressing
- Indexed Addressing

Sub-concepts
Instruction Types

- Data Transfer Instructions
- Arithmetic Instructions
- Control Instructions

Performance Factors

- Memory Access Time
- Instruction Size
- Execution Speed

Relationships (Linking Words)

- Immediate Addressing stores the operand within the instruction itself.
- Direct Addressing retrieves data from a specified memory location.
- Register Addressing accesses operands stored in CPU registers.
- Indirect Addressing uses a register or pointer to locate memory.
- Indexed Addressing combines a base address with an index for accessing arrays.
- Addressing Modes influence memory access time and execution efficiency.
- Data Transfer Instructions move data between registers and memory.
- Arithmetic Instructions perform mathematical and logical operations using different addressing modes.
- Control Instructions manage the program flow through branching and looping operations.

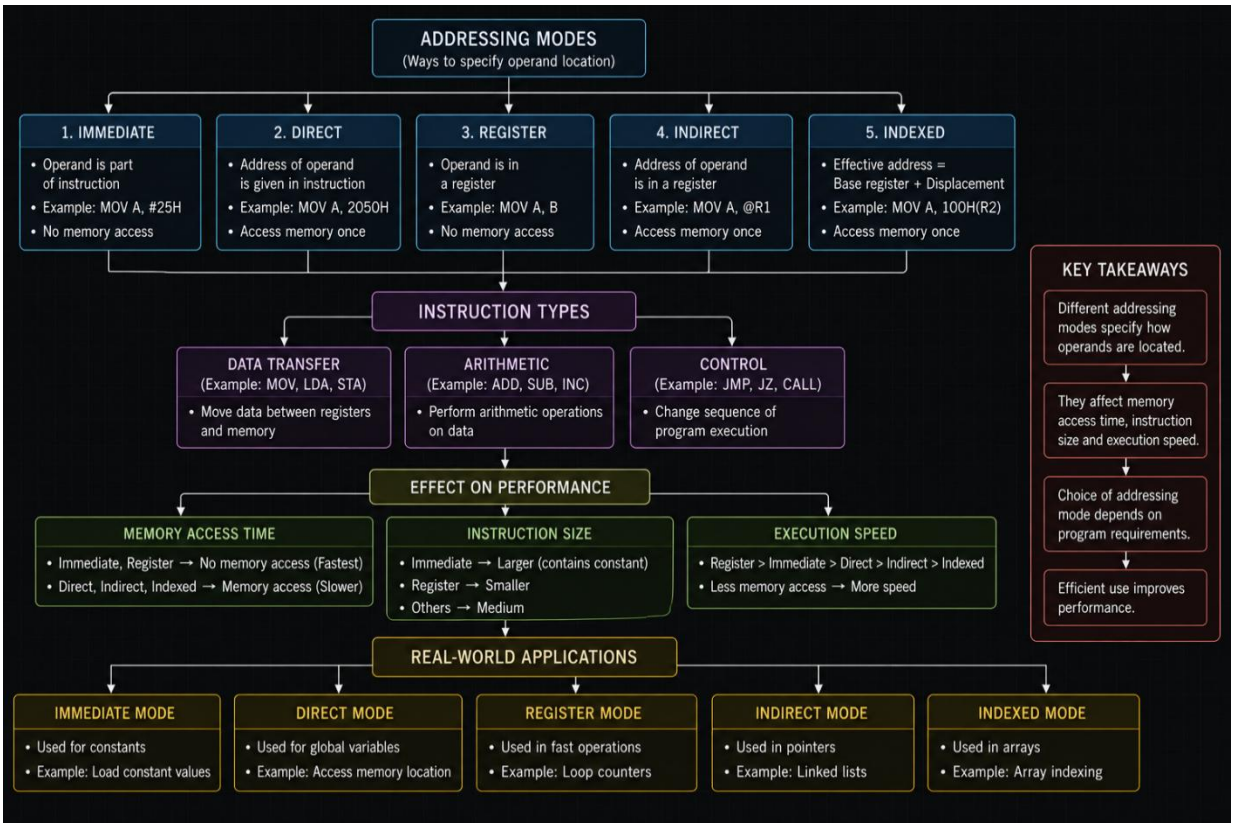
Hierarchy (General → Specific)
Addressing Modes
→ Immediate, Direct, Register, Indirect, Indexed
→ Instruction Types (Data Transfer, Arithmetic, Control)
→ Performance Factors (Memory Access Time, Instruction Size, Execution Speed)
Cross-links

- Register Addressing provides the fastest instruction execution.
- Indexed Addressing simplifies array and string processing.
- Indirect Addressing supports pointer-based and linked data structures.
- Memory Access Time directly affects execution speed and CPU performance.

Real-world Application
Array processing, database management systems, pointer manipulation, embedded systems, operating systems, and compiler design.

Summary / Inference

Choosing the appropriate addressing mode enhances processor efficiency by reducing memory access time, improving execution speed, and optimizing program performance for different computing applications.



5. Number Systems, Memory Storage and Debugging

Central Topic

Number Systems

Organization

Main Concepts

- Binary Number System
- Hexadecimal Number System
- Instruction Representation
- Memory Storage
- Debugging

Sub-concepts

Binary Number System

- Machine Language
- Bit Representation

Hexadecimal Number System

- Compact Representation
- Memory Address Representation

Memory Storage

- Binary Data Storage
- Address Allocation

CPU Execution

- Instruction Fetch
- Instruction Decode
- Instruction Execution

Debugging

- Memory Dump
- Machine Code Analysis

Relationships (Linking Words)

- Binary Number System represents machine instructions using 0s and 1s.
- Hexadecimal Number System simplifies long binary values into a compact form.
- Memory stores instructions and data in binary format.
- CPU fetches, decodes, and executes binary instructions.
- Instruction Representation uses binary encoding for machine operations.
- Hexadecimal represents memory addresses for easier readability.
- Debugging tools display binary data in hexadecimal format.
- Binary values are converted into hexadecimal for efficient analysis.

Hierarchy (General → Specific)

Number Systems

→ Binary Number System, Hexadecimal Number System

→ Instruction Representation

→ Memory Storage

→ CPU Execution (Fetch, Decode, Execute)

→ Debugging (Memory Dump, Machine Code Analysis)

Cross-links

- Binary data is converted into hexadecimal for better readability.
- Memory addresses are commonly represented in hexadecimal notation.

- CPU processes instructions stored in binary format.
- Debuggers use hexadecimal values to examine memory contents and machine code.

Real-world Application

Assembly language programming, embedded systems, operating system development, microprocessor programming, hardware debugging, and firmware development.

Summary / Inference

The Binary and Hexadecimal Number Systems are fundamental to computer architecture. Binary is used for instruction execution and memory storage, while hexadecimal provides a compact and readable representation for memory addresses and debugging, improving program analysis and system development.